CS4246 Snake Agent

Team: P03

Chong Yew Mun Luke	A0245763X	E0905349
Panav Dua	A0246189R	E0912433
Wang Mingchuan	A0240129N	E0774727
Li Chengyue	A0230326u	E0694448

AY2024/25 Sem2
CS4246/CS5446

Content

Introduction and environment

Single Heuristic Agent

- Greedy

Single RL Agent

- Sparse Reward
- LLM - Reward Shaping
  - paper reviewed https://openreview.net/forum?id=tUM39YTRxH

Introduction

- **Objective:** Explore classical or RL-based solutions for Snake Game
- **Learning focus:** Understand key RL concepts like reward shaping, policy optimization, and training stability
- **Hands-on goals:**
    1. Implement & fine-tune RL agents
    2. Experiment with different reward strategies
    3. Compare RL and classical strategies

Environment

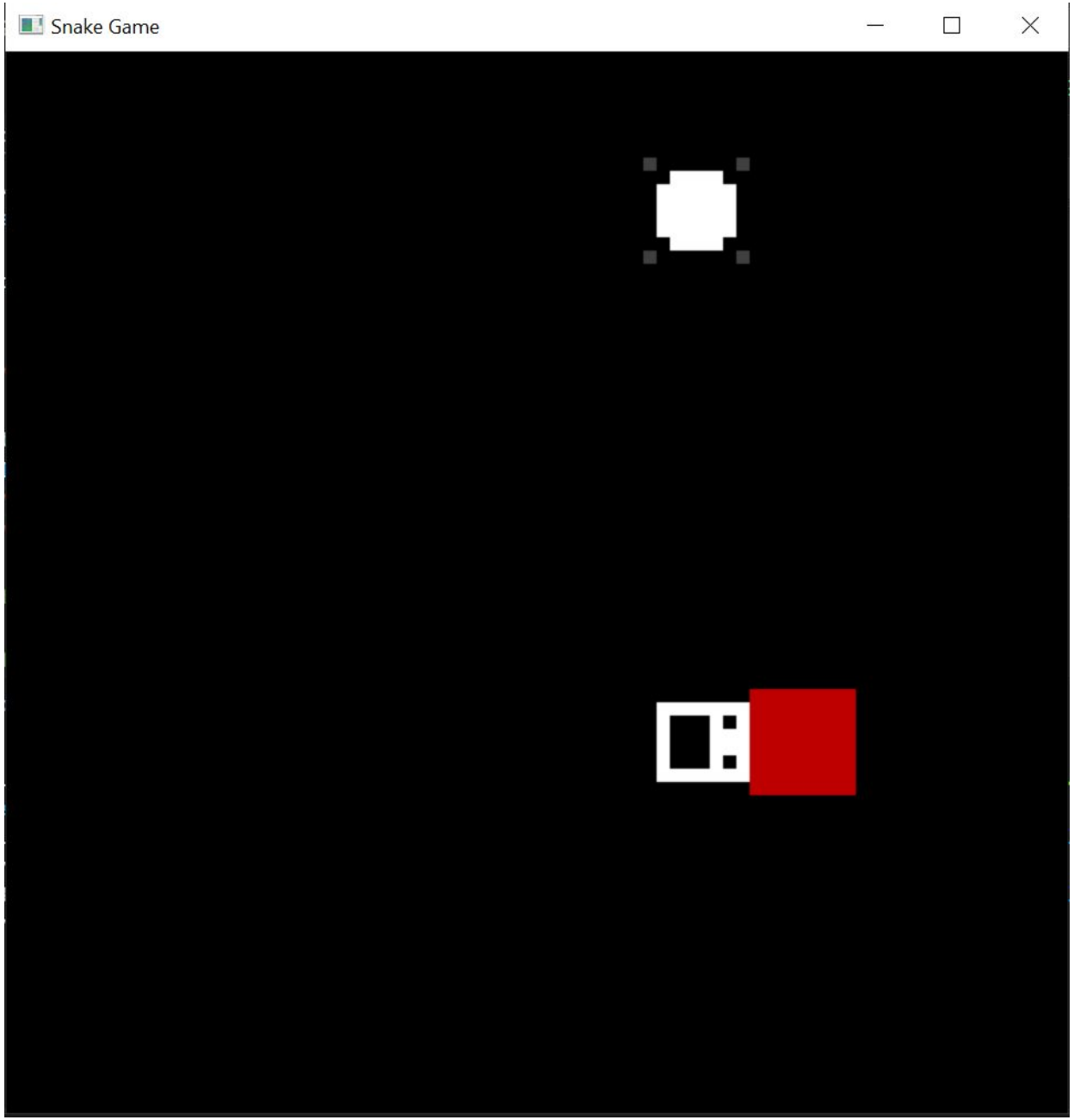
- Used open-source Snake game engine (GitHub)
- Converted it into **Gym-compatible** environment
- Integrated with **Stable Baselines 3 (SB3)** for training





Heuristic Agent

- Discrete
- Deterministic
- Static
- Fully observable



Heuristic Agent

State:

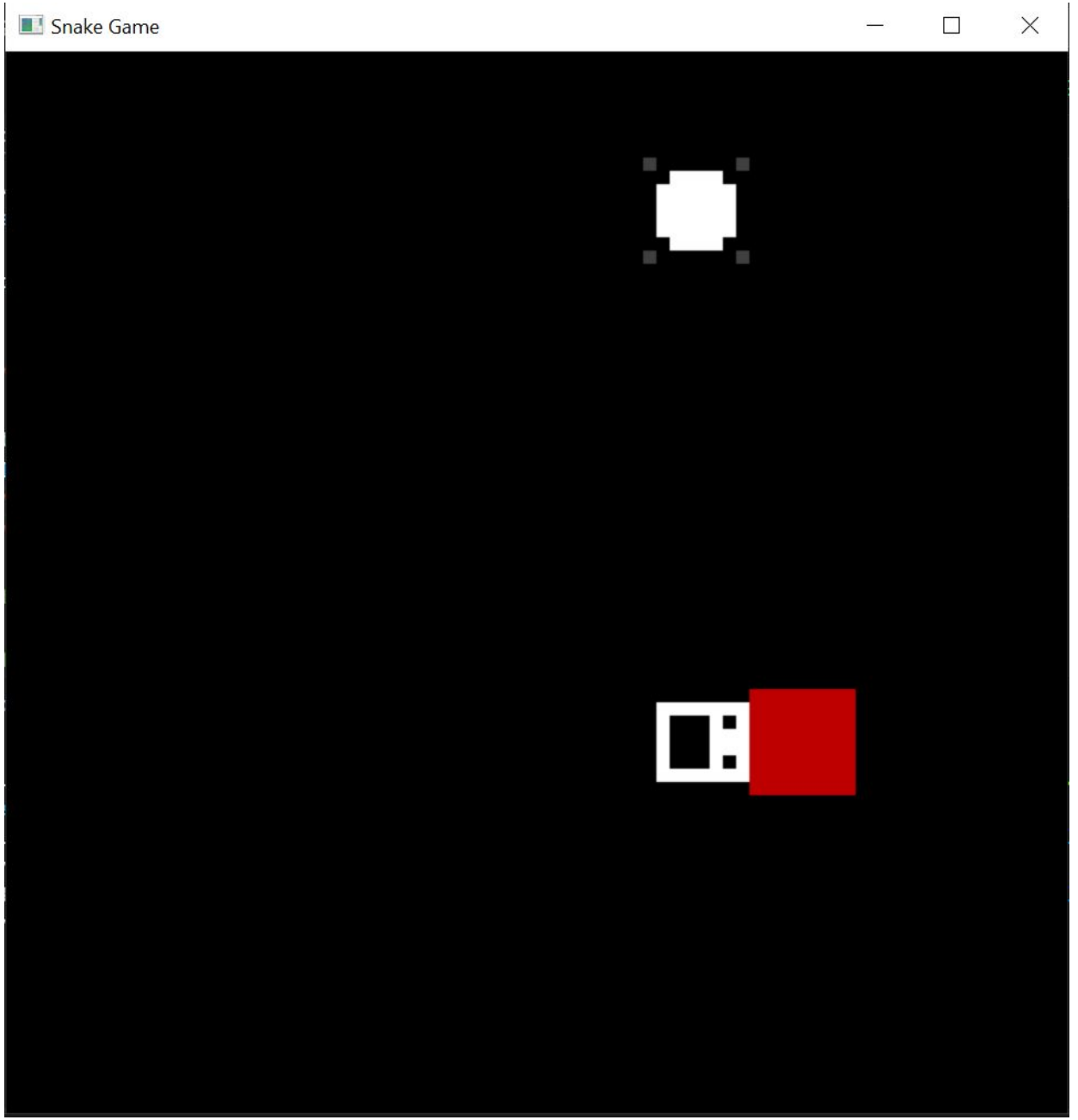
- Location of snake head
- Location of snake tail
- Location of fruit

Actions:

- Up, Down, Left, Right

Goal:

- Head move into fruit
- Head avoid tail and wall



Heuristic Agent 1 (Greedy Best-First Search)

Goal: Move head into fruit

Precondition:

- New head position is not in wall or tail
- Chooses movement (out of 4) closest to the food

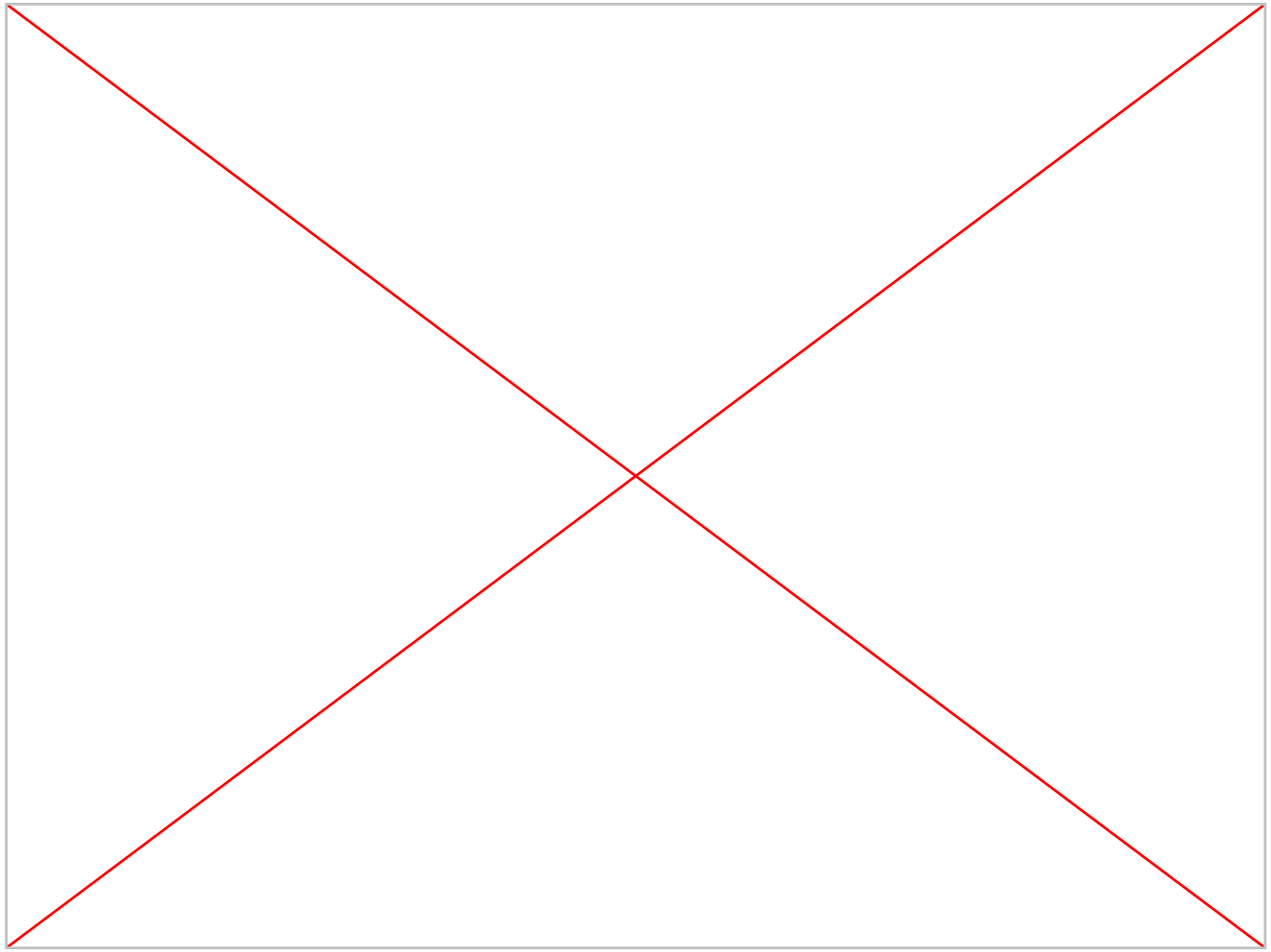
Effect:

- If new head position is empty (not in fruit):
  - New head position -> head
  - Original head position -> tail
  - Last position of tail -> Empty space
- If new head position in fruit:
  - New head position -> head
  - Original head position -> tail

Use a list to keep track of sequence of tail

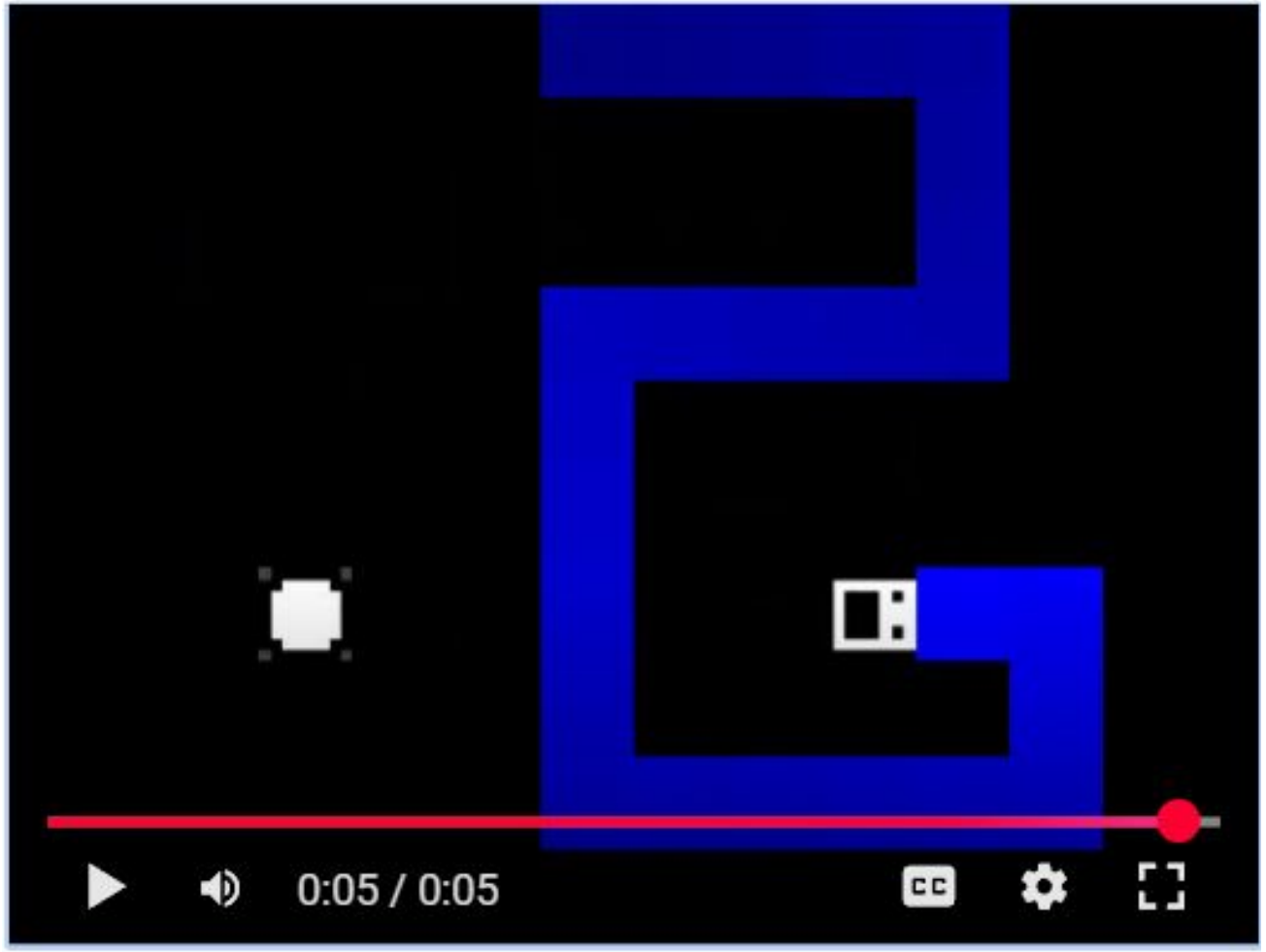
Heuristic Agent 1 (Greedy Best-First Search)

- **Sample Run**



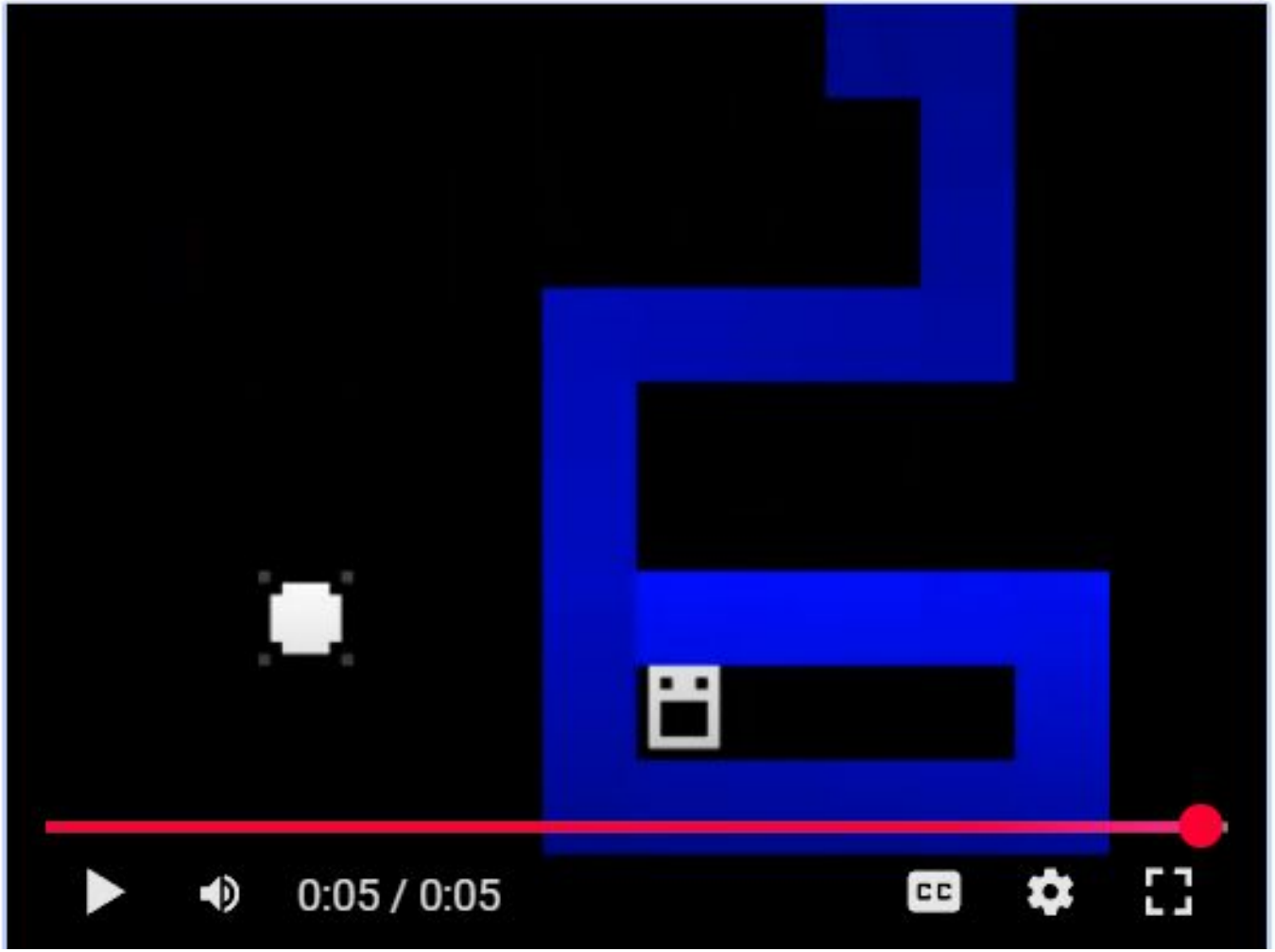
Heuristic Agent 1 (Greedy Best-First Search)

- **Too greedy**
- **Gets trapped**



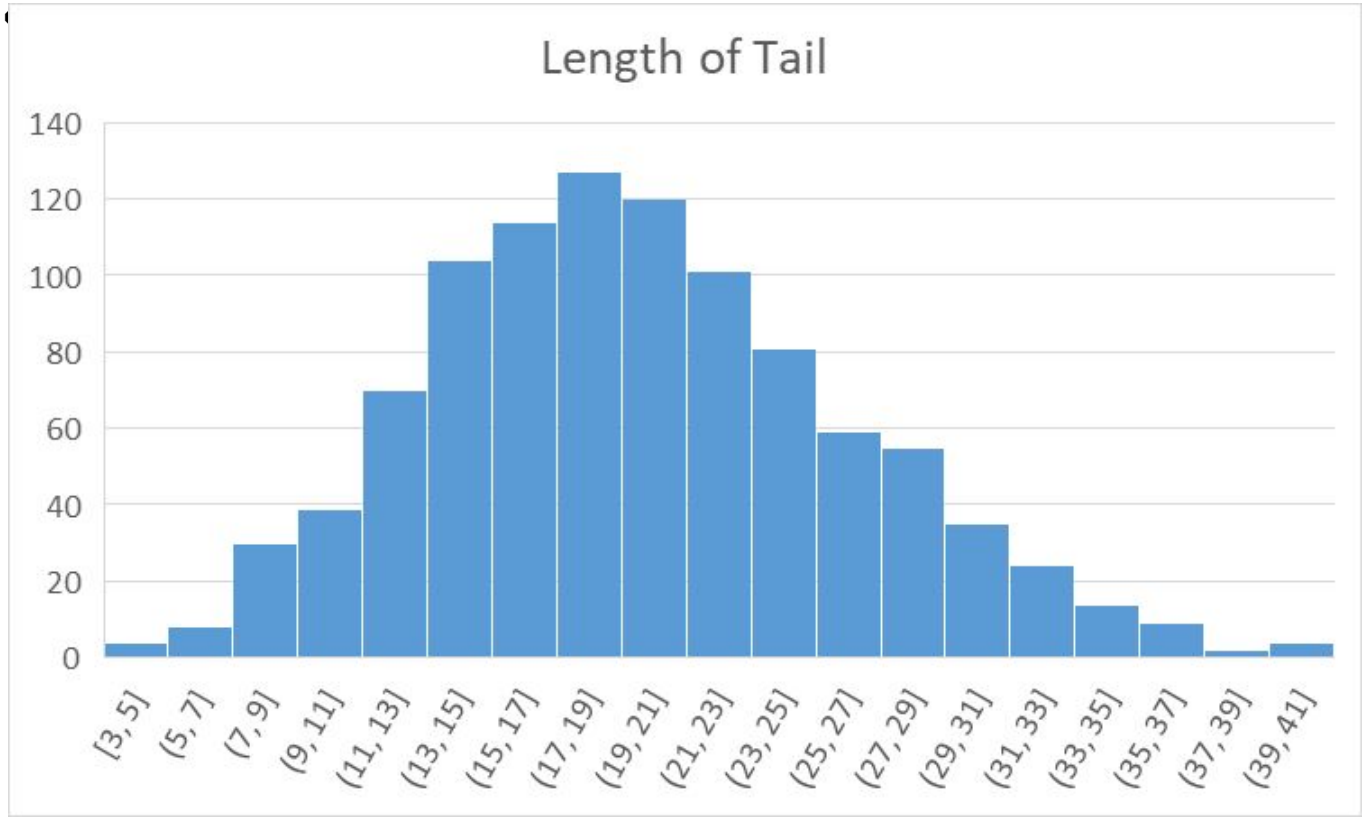
Heuristic Agent 1 (Greedy Best-First Search)

- **Too greedy**
- **Gets trapped**



Heuristic Agent 1 (Greedy Best-First Search)

- **Performance Statistics** (ran a thousand times)



Average score: 20.258

Heuristic Agent 2 (GBFS with area check)

Goal: Move head into fruit

Precondition:

- New head position is not in wall or tail
- Chooses movement (out of 4) closest to the food
- **Does not split the map into 2, if no choice, enter the section with larger area**

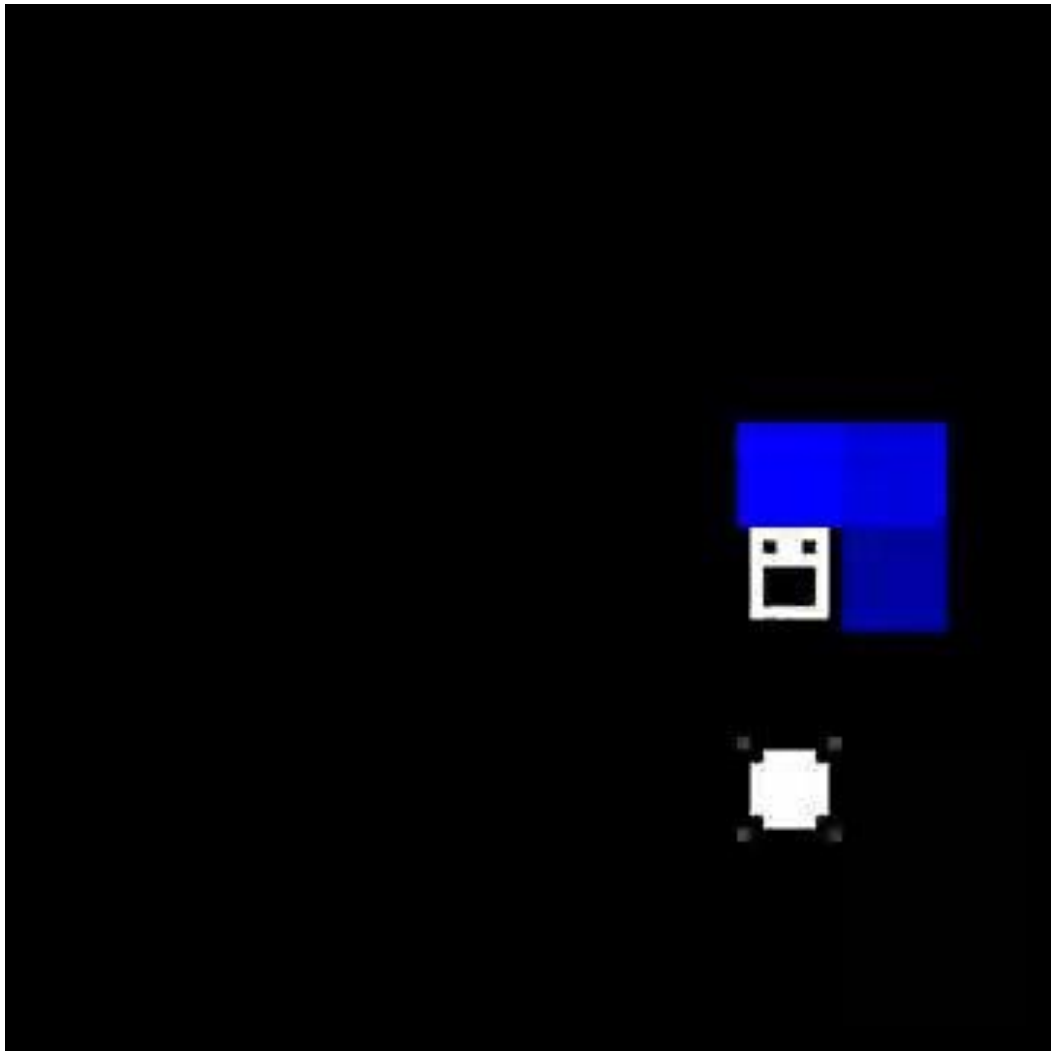
Effect:

- If new head position is empty (not in fruit):
  - New head position -> head
  - Original head position -> tail
  - Last position of tail -> Empty space
- If new head position in fruit:
  - New head position -> head
  - Original head position -> tail

Use a list to keep track of sequence of tail

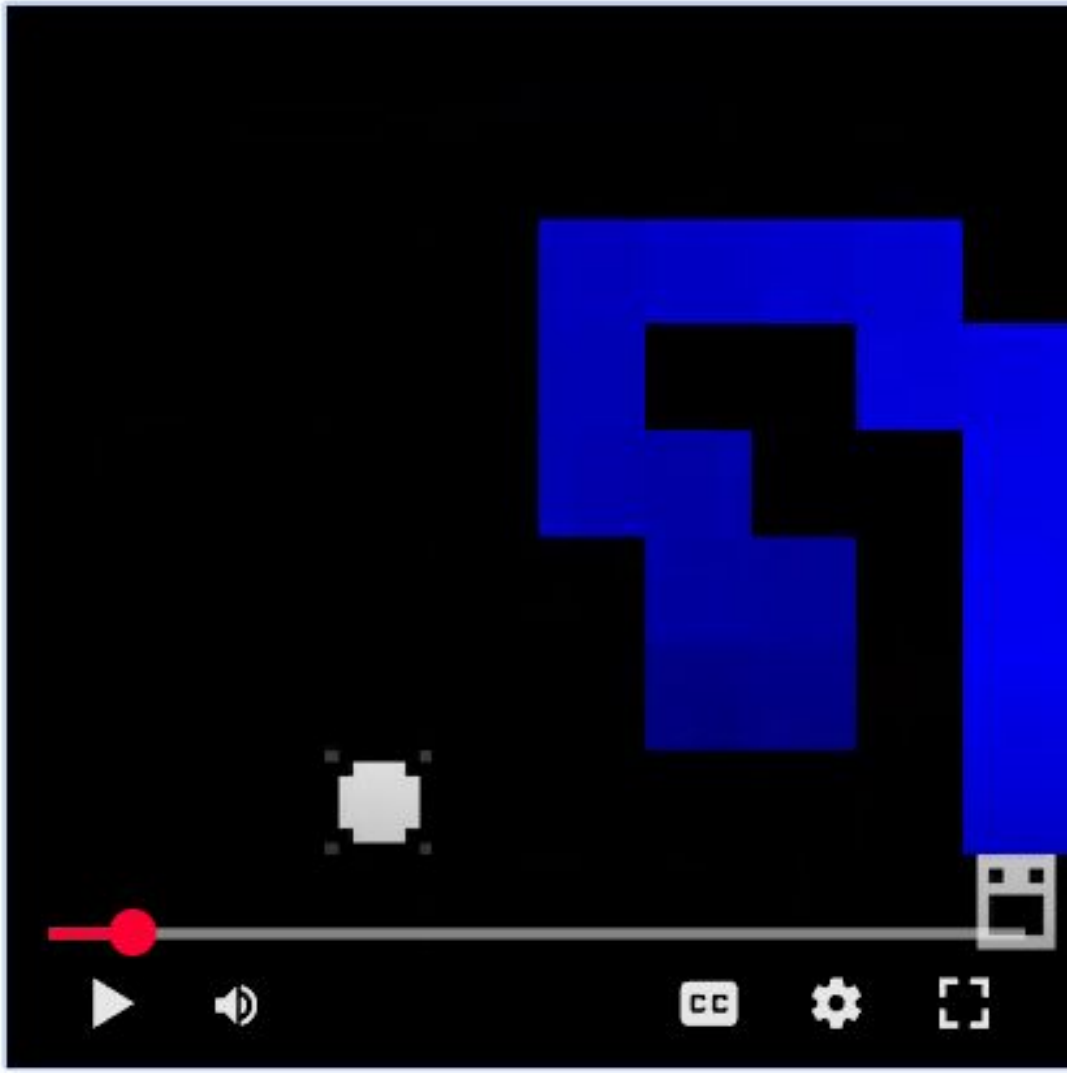
Heuristic Agent 2 (GBFS with area check)

- **Sample Run**

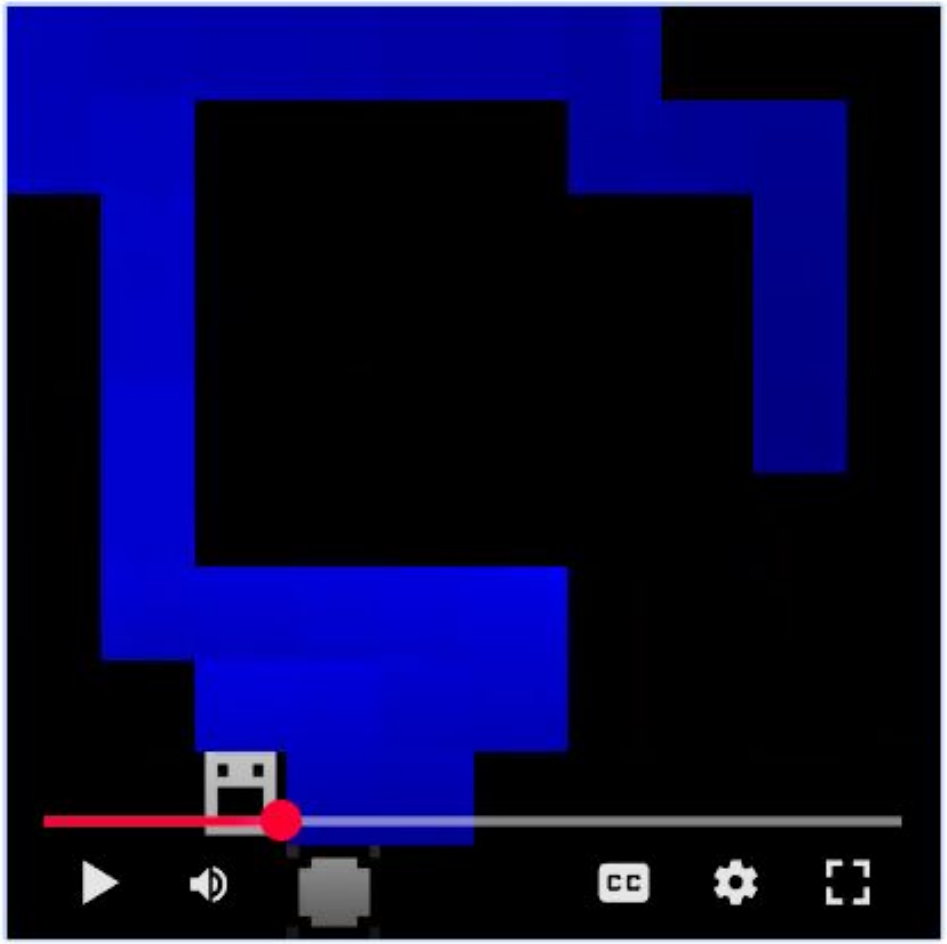


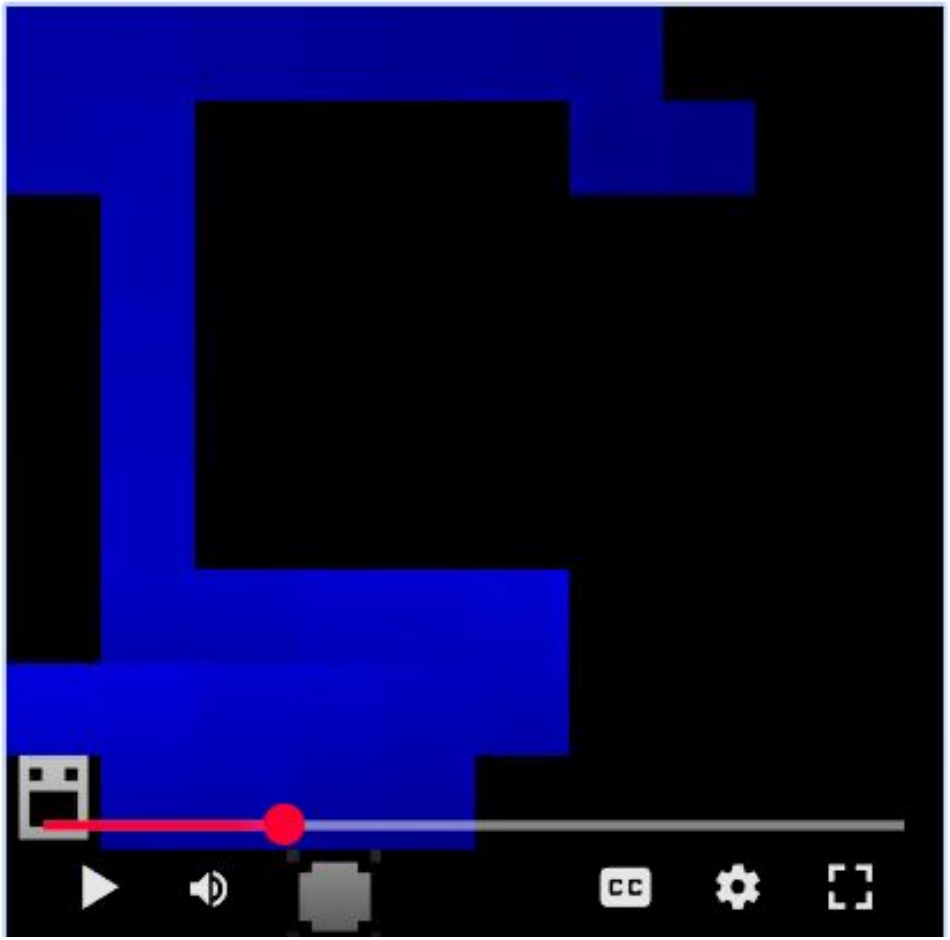
Heuristic Agent 2 (GBFS with area check)

- **Spaces are connected**



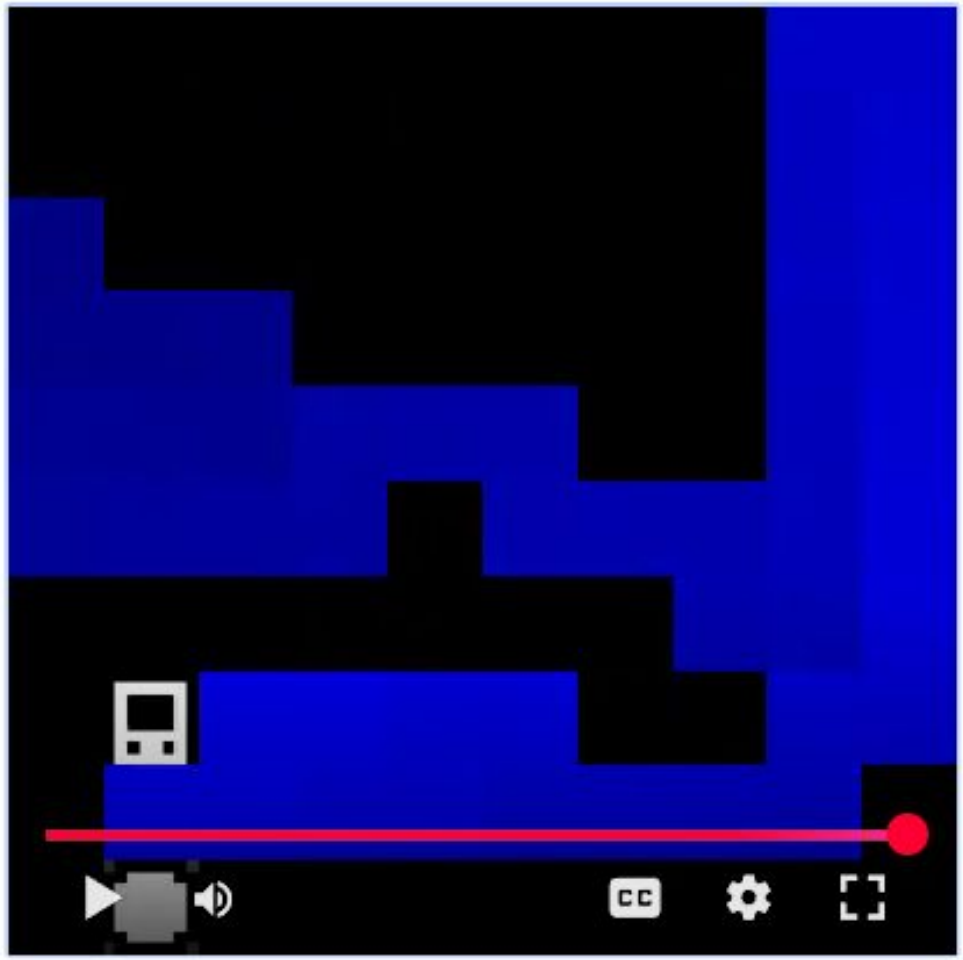
Heuristic Agent 2 (GBFS with area check)





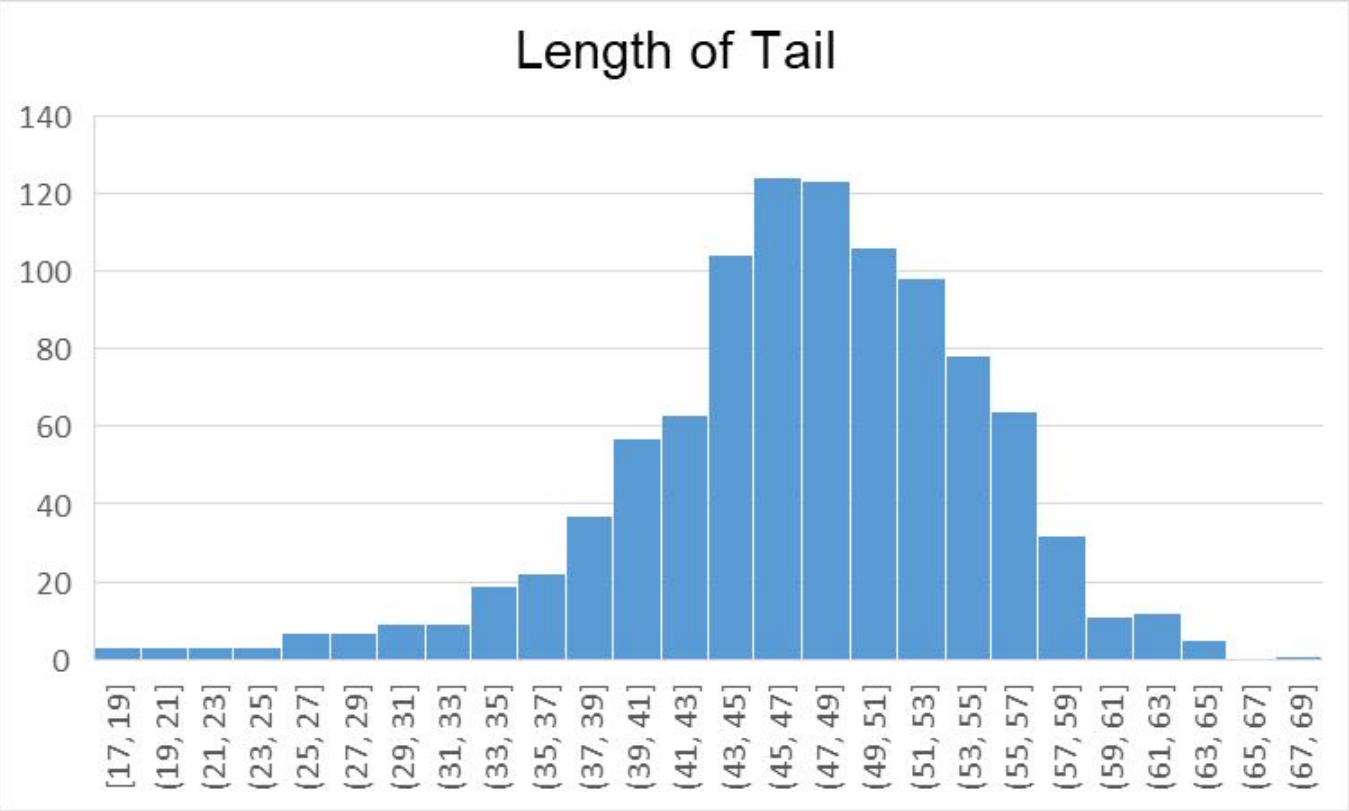
Heuristic Agent 2 (GBFS with area check)

- **Will get stuck**
- **Tail is on the other side**



Heuristic Agent 2 (GBFS with area check)

- **Performance Statistics** (ran a thousand times)



Average score: 47.461

Heuristic Agent 3 (Hamiltonian Cycle (sort of))

Goal: Move head into fruit

Precondition:

- New head position is not in wall or tail
- Chooses movement (out of 4) closest to the food
- **Ensure that the tail is always not blocked**

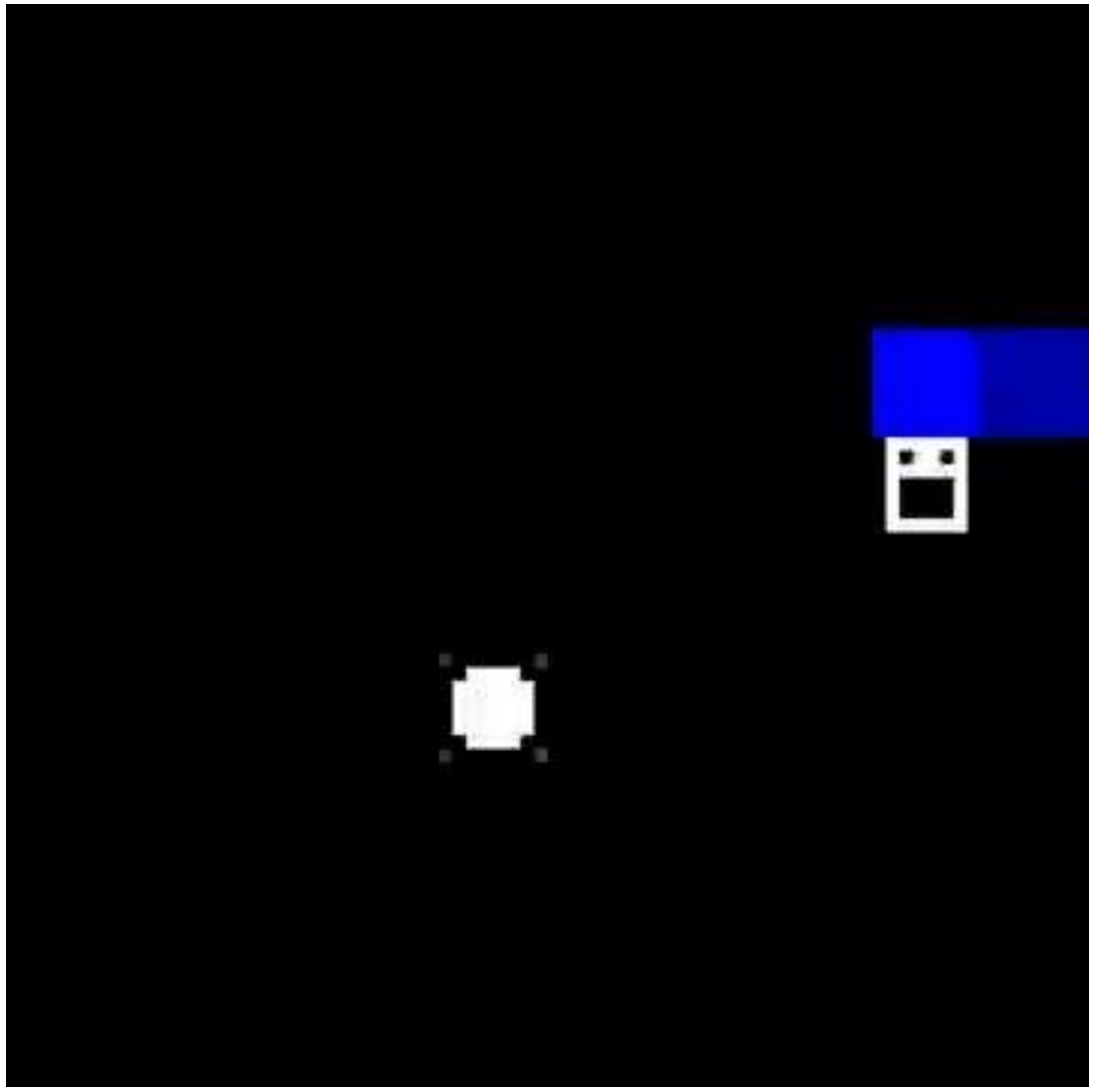
Effect:

- If new head position is empty (not in fruit):
  - New head position -> head
  - Original head position -> tail
  - Last position of tail -> Empty space
- If new head position in fruit:
  - New head position -> head
  - Original head position -> tail

Use a list to keep track of sequence of tail

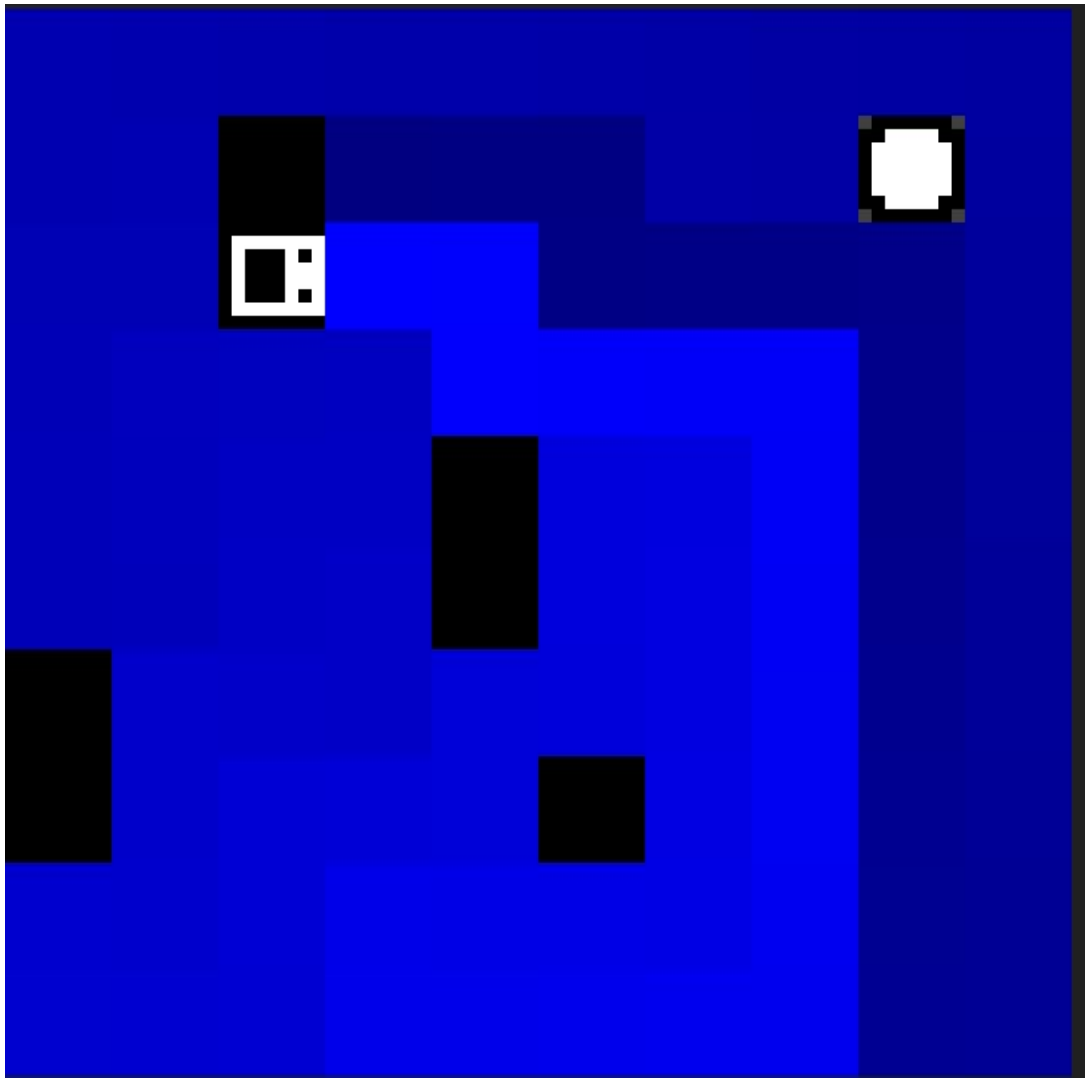
Heuristic Agent 3 (Hamiltonian Cycle (sort of))

- **Run and Problems**



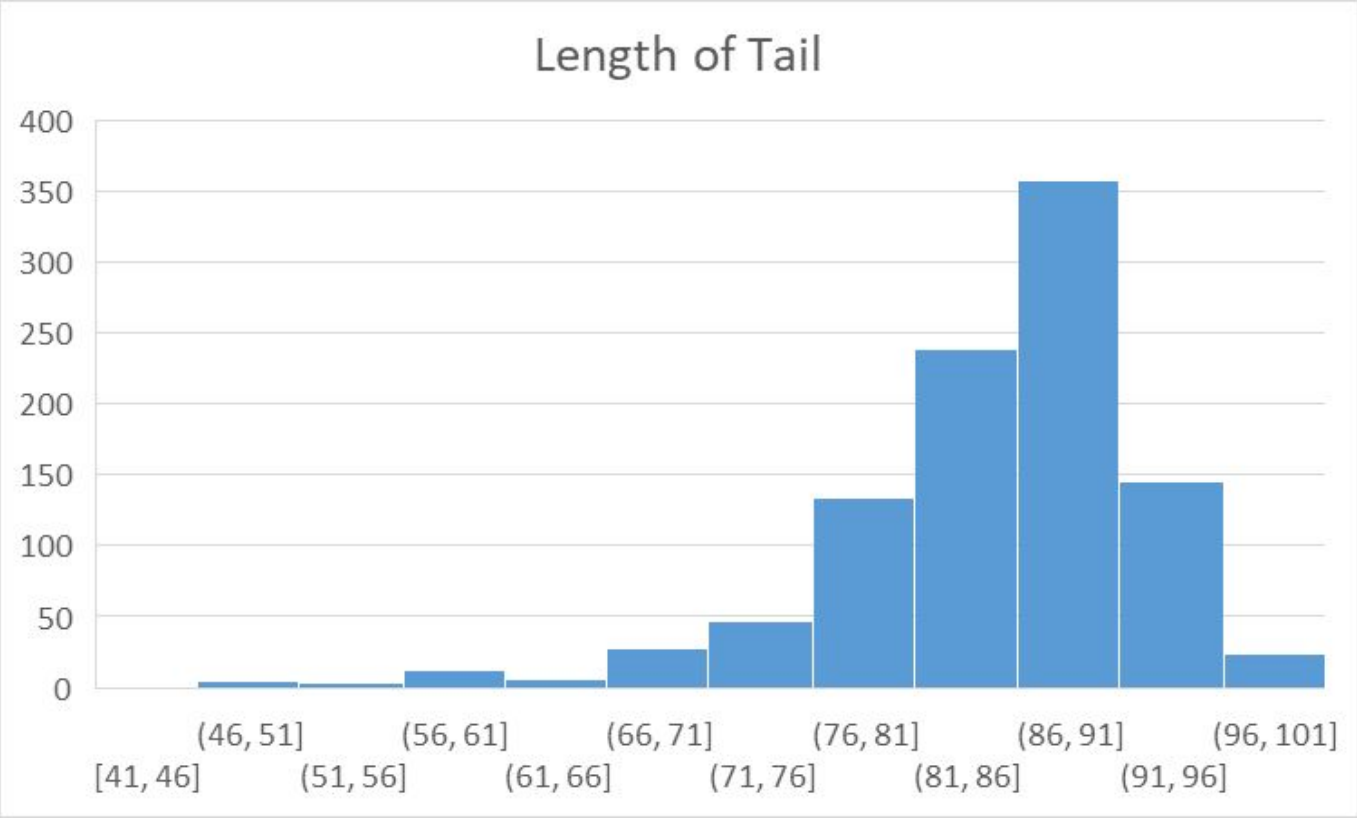
Heuristic Agent 3 (Hamiltonian Cycle (sort of))

- **Almost there!**



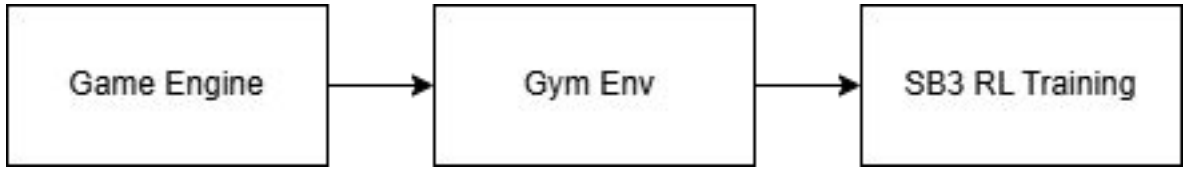
Heuristic Agent 3 (Hamiltonian Cycle (sort of))

- **Performance Statistics** (ran a thousand times)



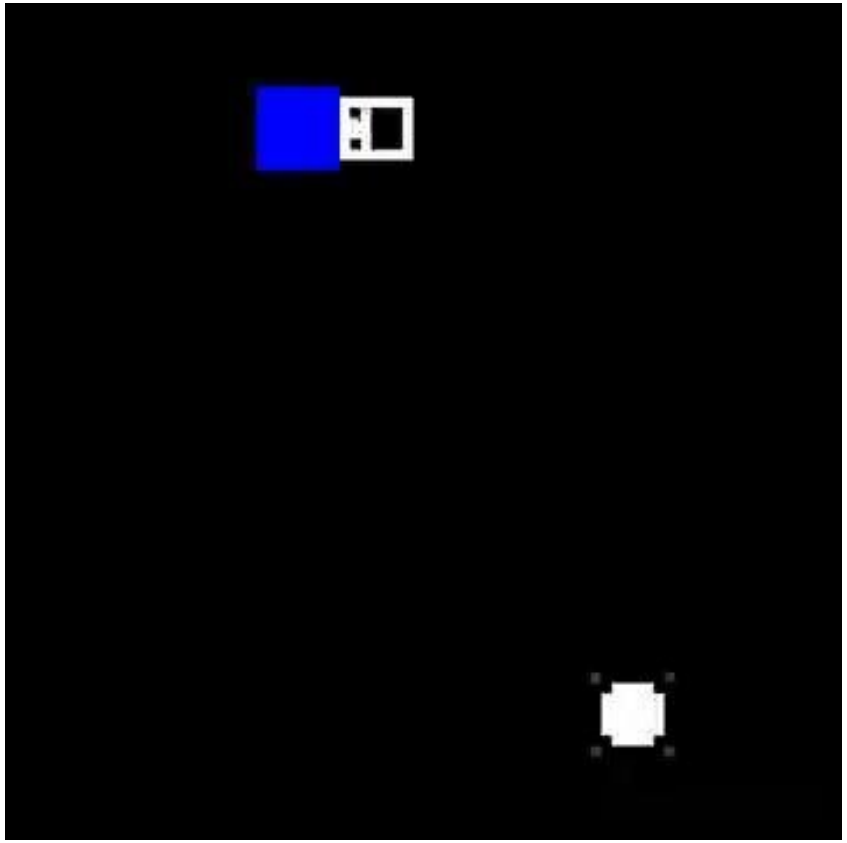
Average score: 85.4
Completed: 24

Environment Setup



A2C algorithm was tested but PPO performs better with the same parameters.

Agent observes images, no additional information



Reinforcement Learning

Domain Specific Variables:

1. Agent rewards - (Dying, Getting Food, Taking a Step)
2. Initial HP, Initial Length, Number of Fruits
3. Action Space - Fixed Perspective or First-person Perspective

Training Variables:

1. Network Architecture
2. Learning rate, GAE Lambda, Entropy Coefficient
3. Episode max length

Domain-Specific Variables

What set of variables would be best for training?

Many changes for testing were made manually - SLOW ITERATION

General Observations:

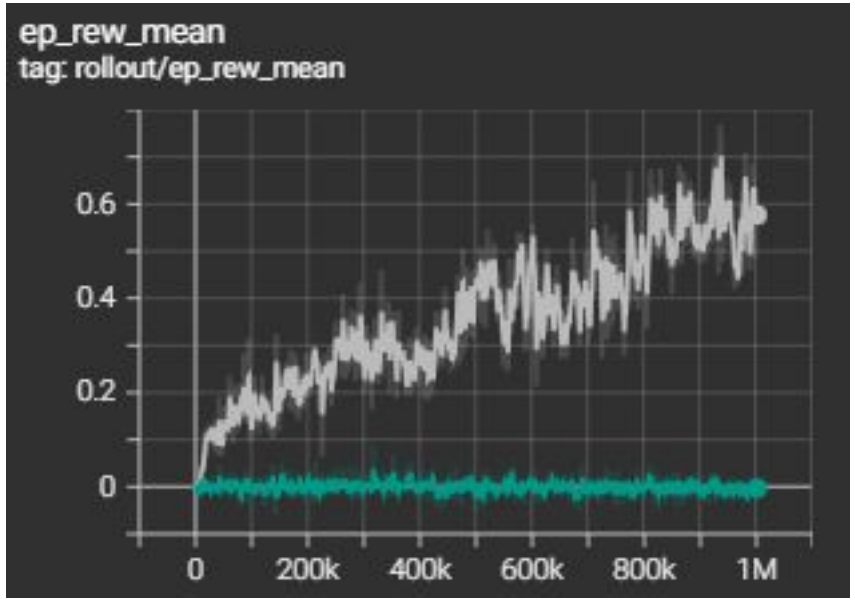
1. Using 4 actions is better, agent doesn't have to learn to differentiate the direction its facing
2. Cannot punish the agent too much for idling, but some punishment necessary
3. Agent learns faster when there are more fruits present

Domain-Specific Variables

Settled Rewards Set After Manual Testing

Rewards were scaled by 0.01

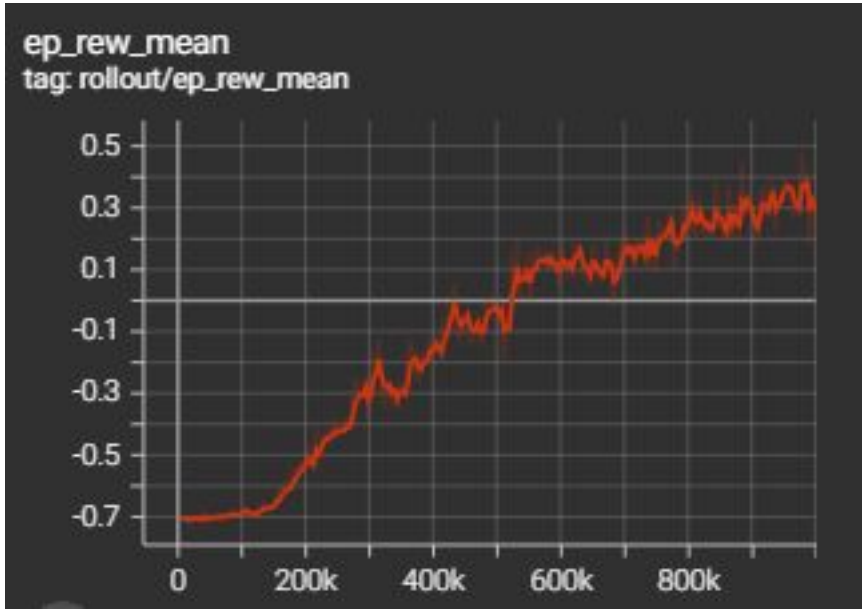
Idling	-0.4, -0.5
Dying	-10
Eating	20
Winning	1



Domain-Specific Variables

Automated Parameter Search (Genetic Algorithm)

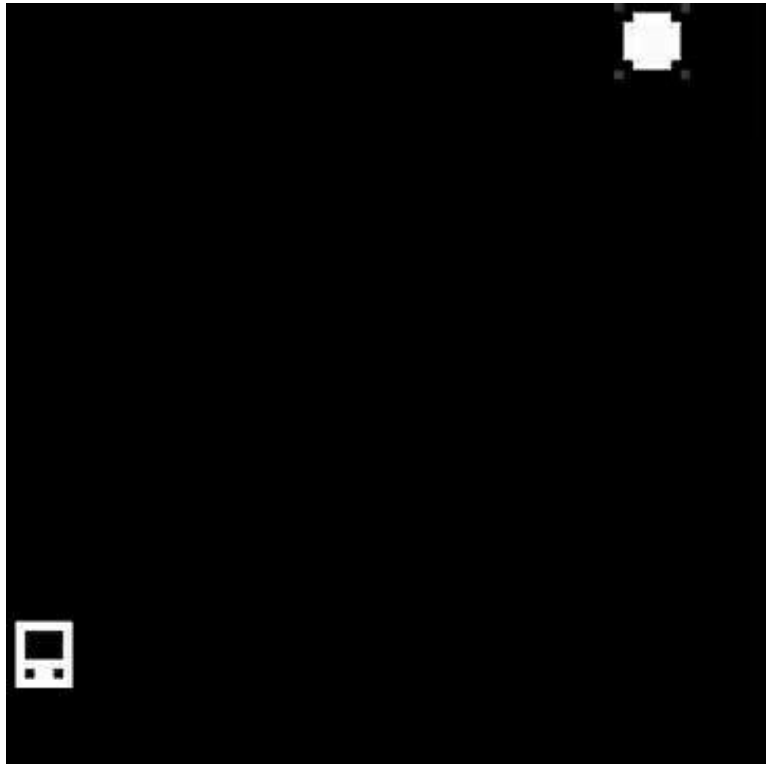
Status	Solution	Search Range
Idling	-1.43	[-3, 0.5]
Dying	-68.2	[-100, -10]
Eating	37.37	[10, 100]



Improvements

Remaining Issues

1. Snake learning too slowly
2. Learning plateaus
3. Not going for rewards



Improvements

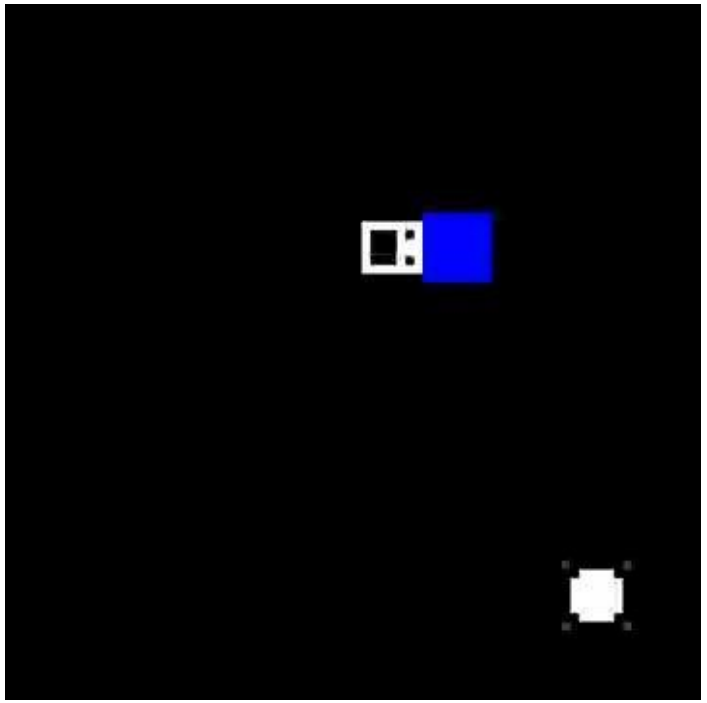
Possible Solutions

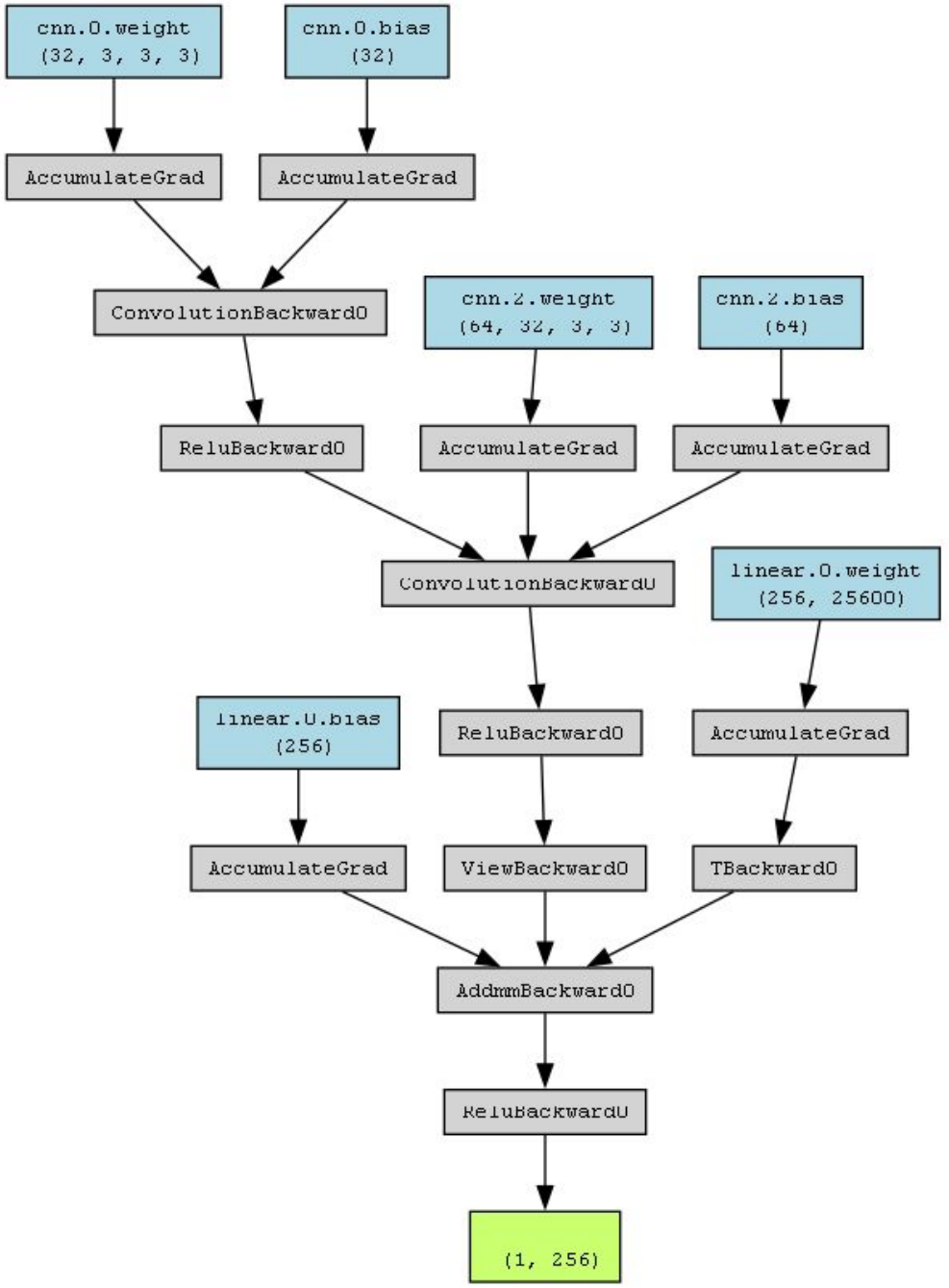
1. Increasing the model size

2. Staged learning with different rewards

Network Architecture

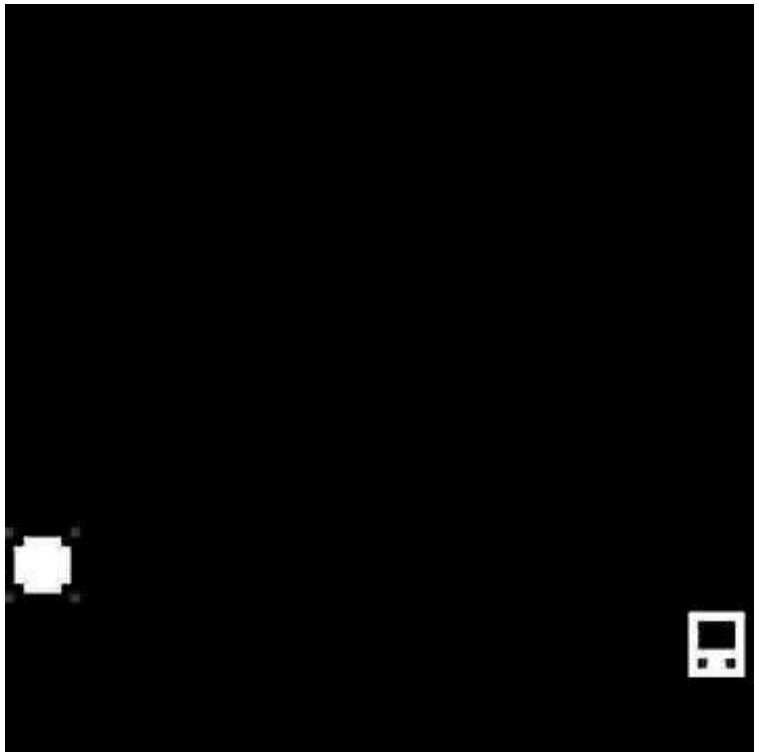
Default Feature Extractor Base:

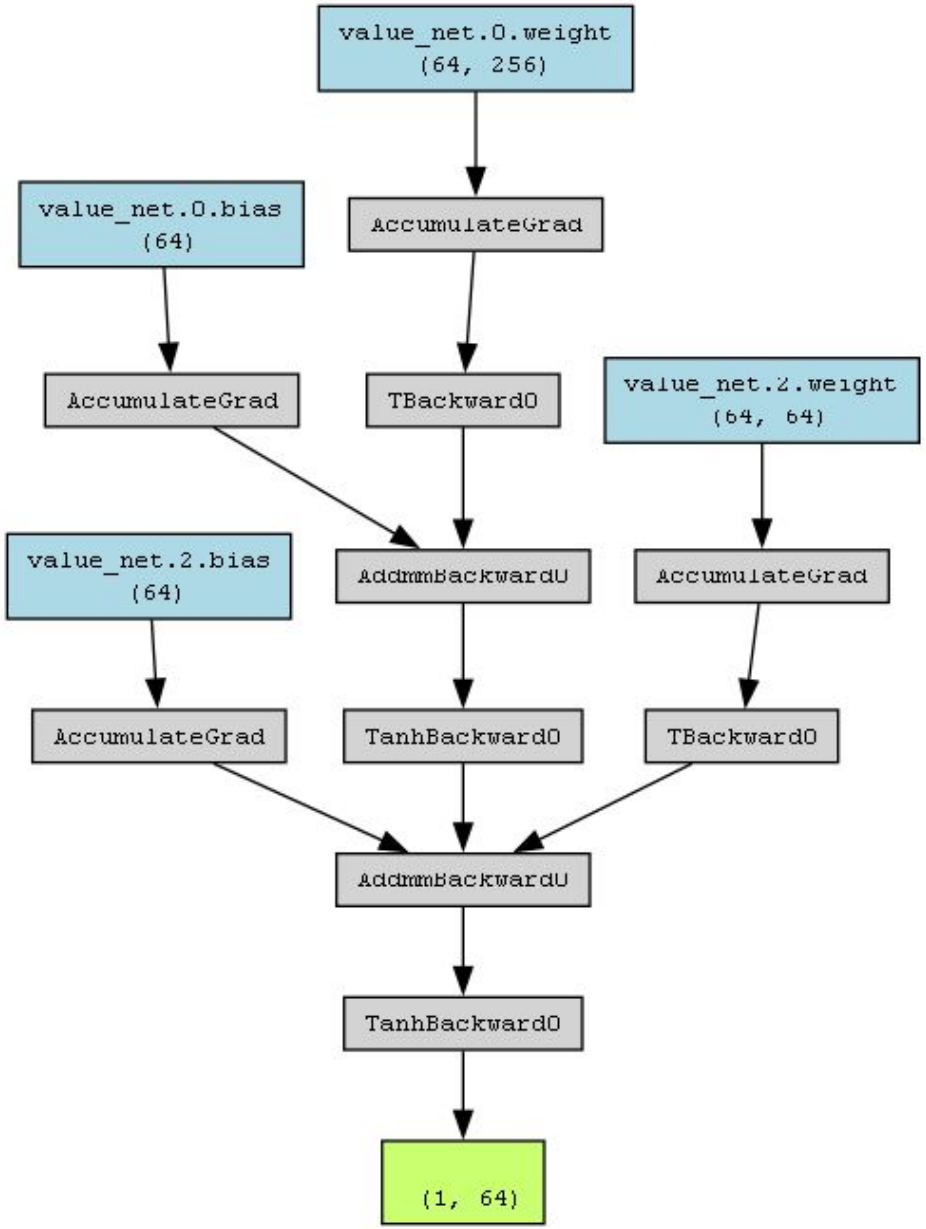




Network Architecture

Our own additional linear layer

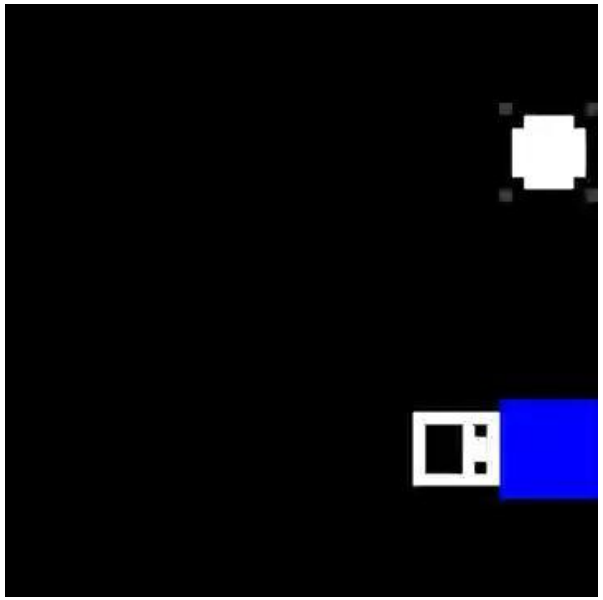




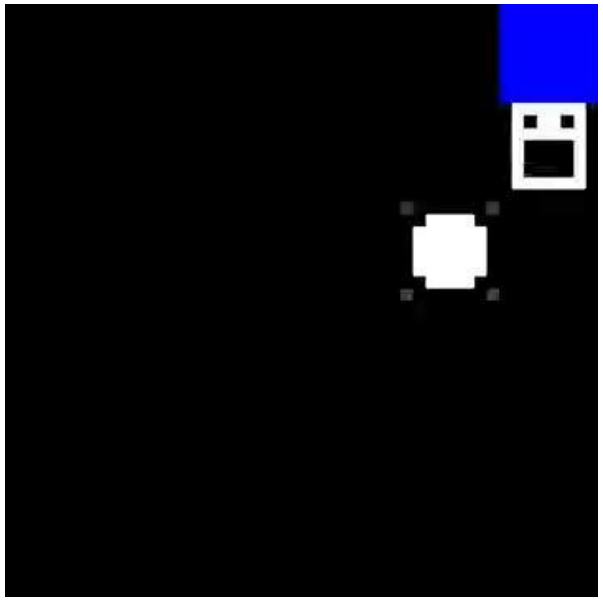
Network Architecture

It solves 6x6! (almost)

Need another idea for improvement



small model



larger model

Network Architecture

Tests were done for changing the policy/value function network, but results largely similar

Staged Learning

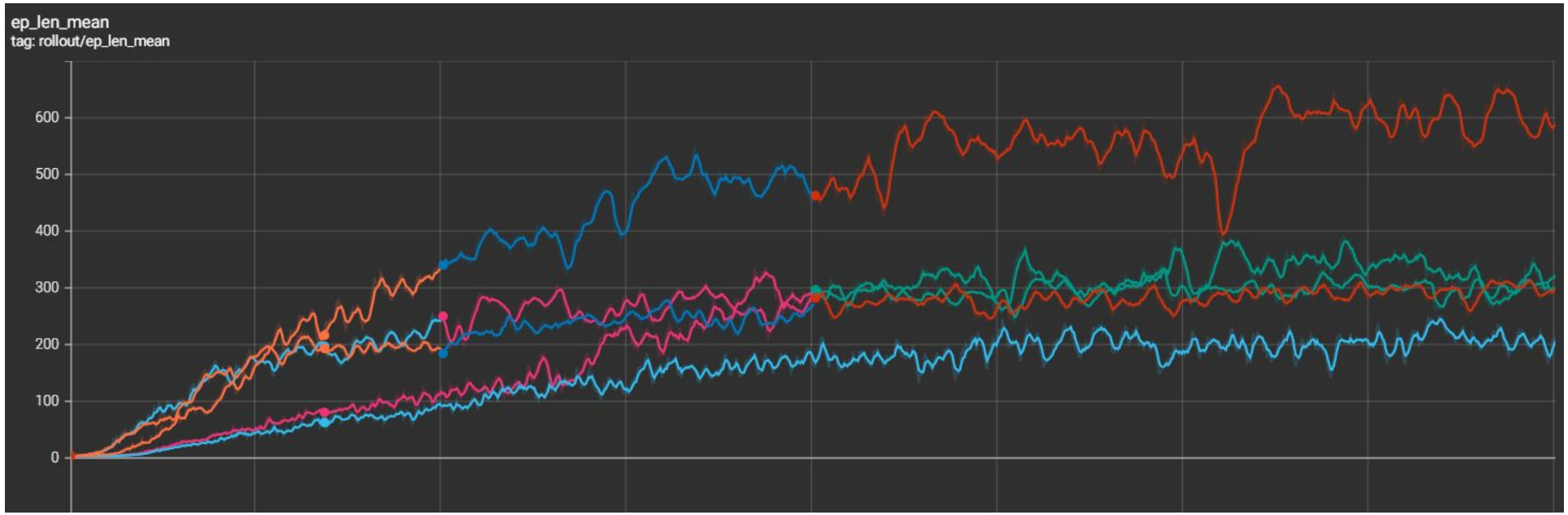
Different reward sets are used in different parts of training

Timesteps	**0 - 1 mil**	**1 mil - 2 mil**	**2 mil - 4 mil**
Eating	-1.43	-1.43	-1.43
Dying	-68.2	-68.2	-68.2
Idling	37.37	37.37	37.37
Winning	1	1	1
Num fruits	10	5	1

Staged Learning

Training Curves

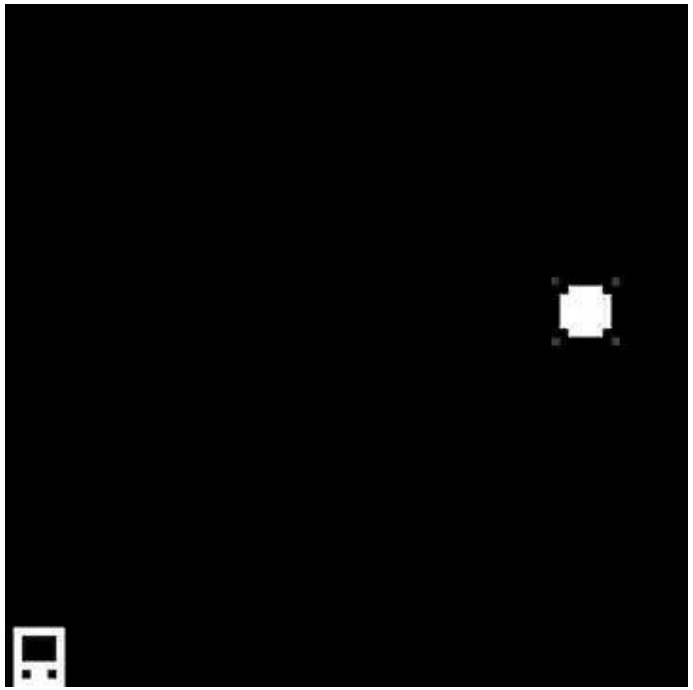
Best run (Top), Without Staging (Bottom), Average runs (Middle)



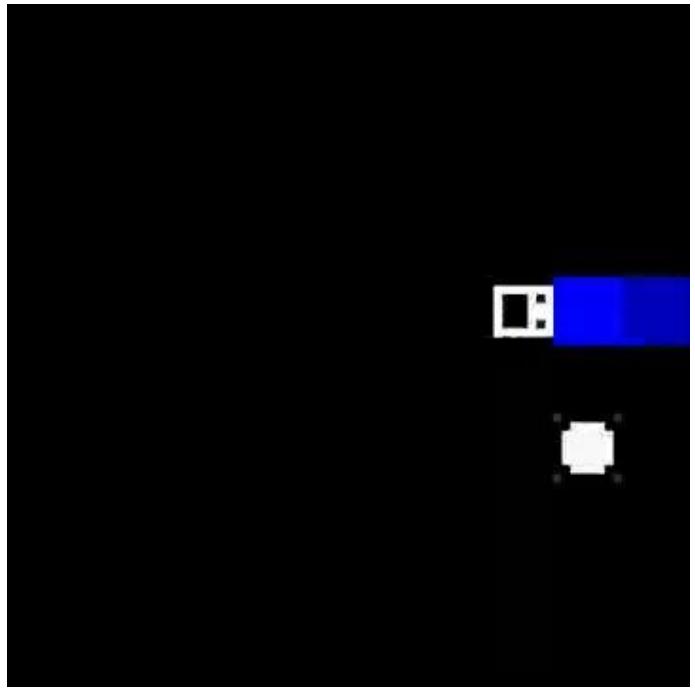
Staged Learning

Different sets were tested but num_fruits was the most promising/stable

Average run



Best run



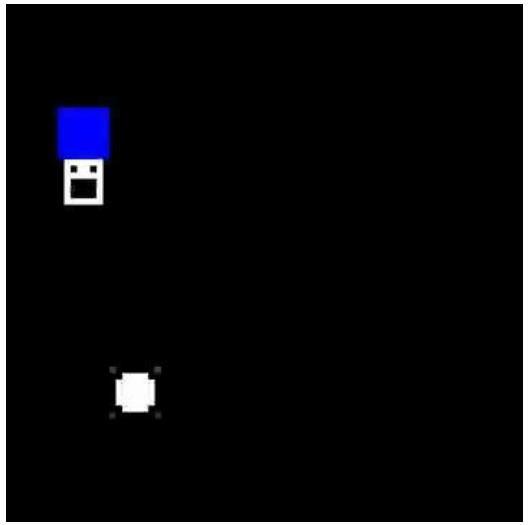
General Observations

Replicating the best run is difficult

Risk massive performance during extra training

Once agent finds “optimal” looping strategy, hard to break out of it





Paper Review - LLMs for Reward Shaping

Paper: *Reward Design with Language Models* (Kwon et al., ICLR 2023)

Core Idea:

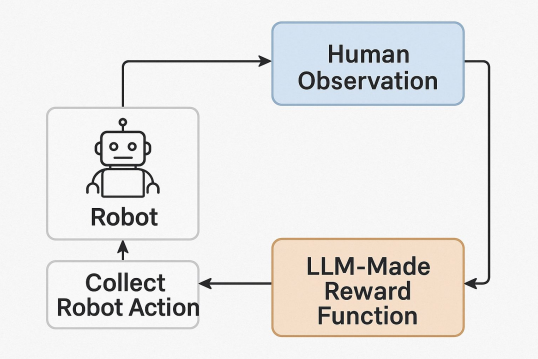
- LLMs can refine reward functions based on post-hoc textual summaries of RL training.
- Allows dynamic reward shaping based on performance descriptions.

Findings:

- LLM-generated rewards can match or exceed hand-tuned baselines.
- Sample efficiency improves due to smarter reward feedback.

Limitations:

- Relies heavily on *natural language* summaries by human.



Our Approach - LLM-Guided Reward Tuning for Snake RL

Goal

Train a Snake RL agent using **LLM-guided reward and parameter tuning** to **avoid plateaus** and **improve sample efficiency.**

Motivation

Manual reward engineering is brittle and inefficient.

LLMs provide generalization and adaptive tuning.

Our Extensions

Structured, **automatically generated metric-based feedback** instead of free-text.

LLM also tunes **PPO hyperparameters, game environment variables** as well as the **rewards**

Tight feedback loop

Train → Analyze → Prompt LLM → Update → Repeat

🧠 LLM acts as an adaptive reward engineer

AY2024/25 Sem2
CS4246/CS5446

Expanded Reward Space

Reward Name	Purpose
food_reward	Basic goal incentive
death_penalty	Strong penalty for failure
step_penalty	Penalize passive behavior
won_reward	Bonus for game completion (if used)
center_bonus	Encourage movement toward map center
loop_penalty	Discourage cyclic paths
wall_follow_penalty	Penalize hugging walls
exploration_bonus	Reward for visiting novel areas
consecutive_food_bonus	Incentivize streaks of food consumption
distance_reduction_reward	Encourage greedy movement toward food
wall_avoidance_bonus	Reward for steering away from walls

Expanded Environment & Parameter Control

entropy_coef (PPO)

Controls exploration vs. exploitation

clip_range (PPO)

Affects policy update stability

num_fruits

Adjusts game difficulty and density

walls_config

Controls map complexity

max_timesteps

Alters episode length, affects learning scale

These can all be tuned by the LLM as part of the config response

LLM Feedback Loop

- Train agent
  for N episodes (e.g., 50000)
  - Extract metrics as LLM input
    Avg food, episode length, entropy, deaths, etc.
    - Generate visualizations as LLM input
      (e.g., performance curves)
      - Build LLM prompt
        Include metrics, reward config, trends, and prior updates
    - LLM evaluates and outputs new config
      Adjust reward weights
      Tune PPO parameters
      Modify wall/fruit count
  - Apply update and continue training

Prompt: "...if metrics show a plateau or instability, update rewards. Otherwise, let current setup run longer..."

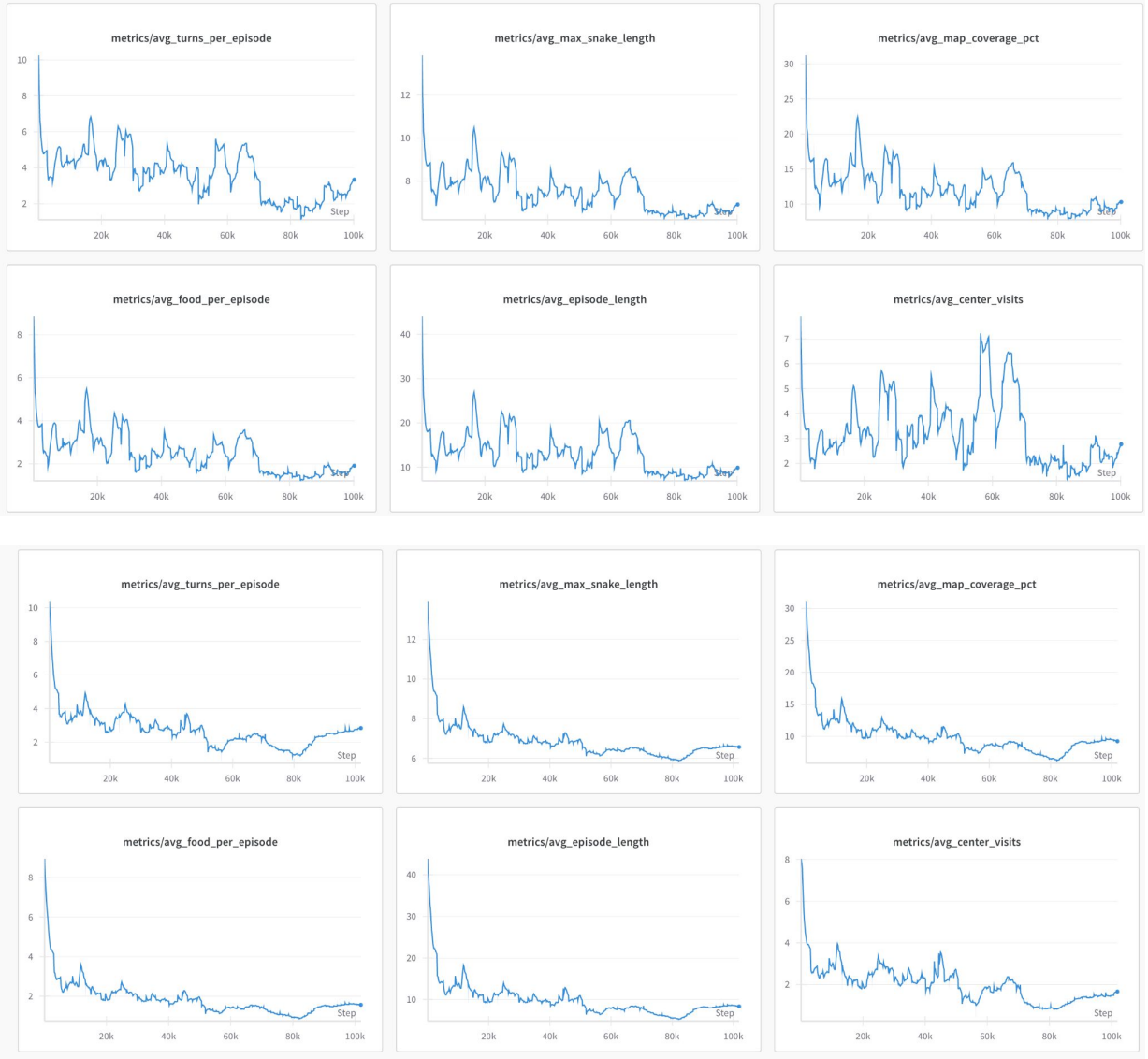
Results: Not so good :(

- Destabilizes training rather than helping, sometimes catastrophic forgetting
- GPT lacks reward tuning ability - might need to fine-tuned for this specific task

Destabilized Training

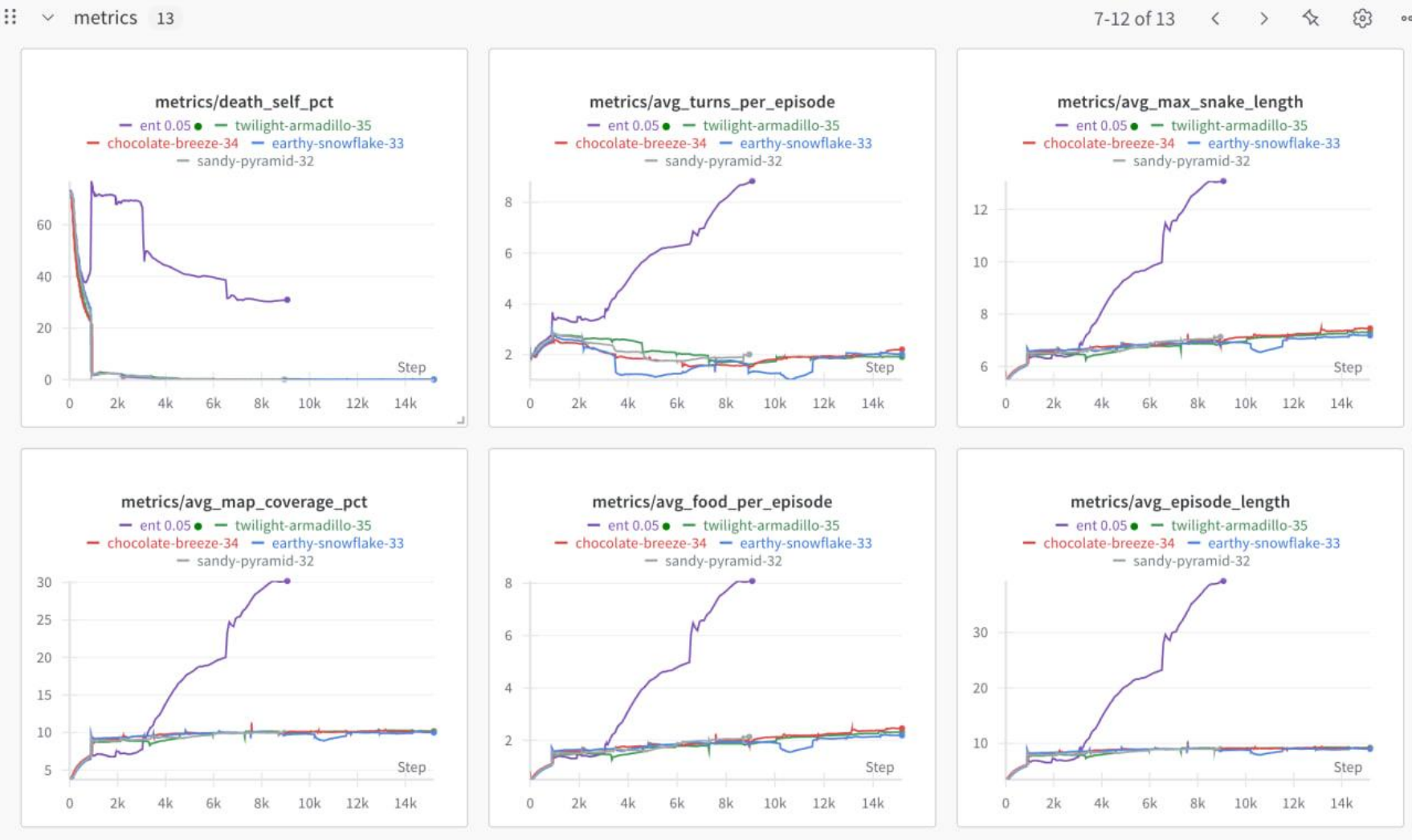


Catastrophic Forgetting



AY2024/25 Sem2
CS4246/CS5446

Occasional Good Case - Entropy Increase



Conclusion

We should use the right tool for the job, RL may not always be the right approach

Training an RL agent has many moving parts, prone to instability

THANK YOU!

References

Snake Game Engine

What Matters In On-Policy Reinforcement Learning? A Large-Scale Empirical Study

PPO Hyperparameter Tuning Recommendations